analysis.ipynb 해설지

사출성형 공정데이터 기반 품질불량 사전예측 및 검사 우선순위 결정
제6회 K-인공지능 제조데이터 분석 경진대회 · 과제 ①

구분	내용
읽는 법	노트북 셀 순서 그대로 [목적] → 코드 → 해설 → **[출력 확인]**(파랑) → **[수정 포인트]**(주황).
구성	0장 전체 구조 · A장 src 모듈 · 1~5장 노트북 셀 해설 35개(보고서 1~4장과 대응) · 부록 1~4 (용어 / 오류 / 체크리스트 / 기초 개념)
처음이라면	부록 4 '기초 개념 정리'를 먼저 훑어본 뒤 0장부터 읽으면 수월하다.
주의	코드 인용은 작성 시점 기준. 코드를 고치면 해설지의 해당 셀도 함께 갱신할 것.

0. 전체 구조 한눈에 보기

프로젝트는 **로직(src/)** 과 **실행 진입점 두 개**로 나뉜다. 노트북은 계산을 새로 짜지 않고 src 함수를 불러 결과를 보여주기만 하므로, 로직을 고치면 run_all.py 와 노트북 양쪽에 동시에 반영된다.

파일	역할	언제 고치나
src/config.py	경로, 시드, CV 횟수, 검사 시나리오	실험 속도·설정 변경
src/data.py	적재, 사출·캐비티 복원, 진단, 입력 X 생성	전처리·특성 추가
src/models.py	비교 모델 정의	모델 추가·하이퍼파라미터
src/evaluate.py	그룹 CV, 임계값, 지표	지표·검증 방식 변경
src/experiments.py	누수 비교, 비라벨 진단, 군집, 오토인코더, 원인 인자	부가 실험 추가·조정
run_all.py	전 과정 자동 실행 → outputs/ 저장 (재현성 10점)	거의 안 고침
analysis.ipynb	결과 시각화·해석 (보고서 그림·표 출처)	그래프·분석 추가

실행 흐름

```
data/*.csv
   |  load_all()  — 사출(shot)·캐비티 구조 복원
   ▼
진단(diagnose) ──────────────────────────────────────────▶ outputs/data_diagnosis.csv
   |  holdout_split()  — 사출 단위 80/20 분리
   ├── dev(80%) — grouped_cv(모델7 × 특성셋2) ─▶ outputs/cv_folds.csv, cv_summary.csv, oof_dev.csv
   |              └ PR-AUC 최고 조합 = 최종모델
   ├── test(20%) — 최종모델 예측 + 원인 인자 ──▶ outputs/test_predictions.csv, test_metrics.json
   └── 부가 실험 (RUN_EXTRA) ──────────────────▶ leakage_cv, unlabeled_domain_auc, self_training_cv,
                                                 regime_clusters, autoencoder_cv (.csv)

analysis.ipynb : 위 결과 파일 + src 함수로 그림/표 생성
```

핵심 설계 3가지 (보고서에서 반복해서 강조할 것)

- **2캐비티 구조 → 사출 단위 그룹 CV**: 같은 사출의 두 제품(공정값 동일)이 학습/평가에 나뉘면 성능이 부풀려진다.
- **임계값은 학습 fold 내부에서만 선정**: 평가 데이터로 임계값을 고르면 F1 이 과대평가된다.
- **PR-AUC 주지표 + Recall@k%**: 불량 1~2% 의 극단적 불균형 → 정확도는 무의미, 검사 우선순위로 바로 연결되는 지표 사용.

A. src 모듈 해설

노트북 셀을 이해하려면 먼저 이 다섯 파일이 무엇을 하는지 알아야 한다. 전체 코드는 각 파일을 직접 열어 확인한다.

src/config.py — 경로·상수

- 모든 설정값이 한 곳에 있다. CV 반복 횟수(N_REPEATS), fold 수(N_SPLITS), 테스트 비율(TEST_FRAC), 검사 시나리오(TOP_K).
- **RUN_EXTRA**: 부가 실험 실행 여부(기본 True). **SEMI_REPEATS**: 자기학습 실험 반복 수(기본 3, 비라벨 7만 행 예측이 있어 느림).
- **수정**: 빠르게 실험할 때 N_REPEATS 를 5 → 1 로 줄이면 실행 시간이 약 1/5.

src/data.py — 데이터 적재·구조 복원·진단

- **add_shot_structure**: 직전 행과 공정값 차이 합이 0 이면 같은 사출로 본다 → shot_id, cavity 생성. 이 프로젝트의 핵심 전처리.
- **diagnose**: 1장 진단표. **scaling_check**: 라벨/비라벨 표준화 확인.
- **feature_matrix**: 모델 입력 X 생성. 상수 컬럼 제거 + is_rg3(제품 구분) + (선택) cavity.
- **수정**: 새 특성(예: 변수 간 비율·차이)을 추가하려면 feature_matrix 안에서 X['새이름'] = ... 로 넣는다.

src/models.py — 비교 모델

- LR_baseline(표준화+로지스틱), RandomForest·ExtraTrees(배깅), LightGBM·XGBoost·CatBoost(부스팅, 얕은 나무·강한 규제), RF+ET_ensemble(소프트 보팅). 모두 불량 클래스에 가중치를 준다.
- **수정**: 모델 추가는 get_models() 딕셔너리에 '이름': 모델 한 줄 추가 → run_all 과 노트북에 자동 반영.

src/evaluate.py — 평가 로직

- **grouped_cv**: StratifiedGroupKFold(사출 단위 그룹 + 불량 비율 유지) 5-fold × 5회 반복.
- **best_threshold / inner_oof**: 학습 fold 내부 예측으로만 F1 최대 임계값 선정.
- **recall_at_k**: 상위 k% 검사 시 포착률. **metrics**: 지표 묶음 계산.
- **수정**: 지표 추가는 metrics() 딕셔너리에 한 줄 추가.

src/experiments.py — 부가 실험 (run_all.py 6단계)

- **leakage_comparison**: 행 단위 CV vs 사출 단위 그룹 CV PR-AUC 비교 → leakage_cv.csv.
- **domain_discrimination**: 라벨/비라벨 판별 AUC → unlabeled_domain_auc.csv. **self_training**: 가짜 라벨 자기학습 → self_training_cv.csv.
- **regime_clusters**: K-평균 운전구간 군집 → regime_clusters.csv, regime_cluster_assign.csv.
- **autoencoder_cv**: 양품 학습 오토인코더 비교 → autoencoder_cv.csv.
- **shap_positive / top_factors**: 불량 클래스 SHAP (앙상블은 구성 모델 평균) → 제품별 원인 인자 top3 (test_predictions.csv 의 factor_1~3).
- **significance_tests**: 최종모델 vs 나머지 조합 짝 비교 (Wilcoxon, Nadeau-Bengio 보정 t) → significance.csv.
- **imbalance_comparison**: 클래스 가중치 / 가중치 없음 / SMOTE-NC / BorderlineSMOTE / ADASYN 비교 → imbalance_cv.csv.
- **dl_stacking_comparison**: 최종모델 vs 스태킹 앙상블 vs MLP 2종 비교 → dl_stacking_cv.csv.
- **time_block_validation**: 연속 생산 구간 단위 그룹 CV (처음 보는 구간, 시간 누수 점검) → time_block_cv.csv.
- **수정**: 실험을 추가하려면 함수를 만들고 run_all.py 의 run_extra() 에 저장 줄을 넣는다.

핵심 코드: 사출 구조 복원 (src/data.py)

```
cols = process_cols(df)                                  # 공정변수 24개
new_shot = df[cols].diff().abs().sum(axis=1).ne(0)       # 직전 행과 하나라도 다르면 새 사출
df['shot_id'] = new_shot.cumsum() - 1                    # 새 사출마다 번호 +1
df['cavity']  = df.groupby('shot_id').cumcount()         # 사출 안에서 0, 1 순번
```

가정: **행 순서 = 캐비티 순서**. 데이터에 캐비티 번호가 명시되지 않았으므로 보고서에 가정으로 밝힌다.

핵심 코드: 누수 없는 평가 (src/evaluate.py)

```
cv = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED + r)
for tr, va in cv.split(X, y, groups):                    # groups = 제품_사출번호
    m   = clone(model).fit(X.iloc[tr], y[tr])
    thr = best_threshold(y[tr], inner_oof(model, X.iloc[tr], y[tr], groups[tr], ...))  # 학습 fold 안에서만
    p   = m.predict_proba(X.iloc[va])[:, 1]               # 평가 fold 는 예측에만 사용
```

0장. 환경 설정

[셀 1] 0-1. 라이브러리와 설정 불러오기

목적 노트북 전체에서 쓰는 도구와 우리 프로젝트 함수(src/)를 한 번에 불러온다.

```
import json
import warnings
import platform

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.base import clone
import matplotlib.font_manager as fm

from src.config import INDEX_COL, OUT_DIR, SEED, TARGET
from src.data import diagnose, feature_matrix, load_all, load_labeled, process_cols, scaling_check
from src.evaluate import best_threshold, inner_oof, metrics
from src.models import get_models
from run_all import holdout_split

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
sns.set_style("whitegrid")
_installed = {f.name for f in fm.fontManager.ttflist}
KOREAN_FONT = next((f for f in ["Malgun Gothic", "AppleGothic", "NanumGothic", "NanumBarunGothic", "Noto Sans CJ
    K KR"]
                    if f in _installed), None)
plt.rcParams["font.family"] = [KOREAN_FONT, "DejaVu Sans"] if KOREAN_FONT else ["DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False   # 축 눈금의 음수 기호 깨짐 방지
print("그래프 폰트:", plt.rcParams["font.family"])
```

- **src.config**: 경로(OUT_DIR), 정답 컬럼명(TARGET), 시드(SEED) 같은 상수.
- **src.data**: 데이터 적재·사출 구조 복원·진단 함수. **src.evaluate**: 평가 지표 함수. **src.models**: 비교 모델 7종.
- **from run_all import holdout_split**: 테스트셋 분리 방식을 run_all.py 와 똑같이 쓰기 위해 가져온다 (노트북과 스크립트 결과가 어긋나지 않게).
- **설치된 한글 폰트 자동 탐색**: fm.fontManager.ttflist 에서 실제 설치된 폰트 이름을 모아(_installed), 후보 목록(맑은 고딕 → AppleGothic → 나눔고딕 → 나눔바른고딕 → Noto Sans CJK KR) 중 처음 발견되는 것을 KOREAN_FONT 로 쓴다. 운영체제가 아니라 '실제로 있는 폰트'를 기준으로 고르므로 Windows·Mac·KAMP-NOTE 어디서든 동작한다.
- **기호 대체 폰트**: font.family 를 [한글 폰트, 'DejaVu Sans'] 목록으로 준다. matplotlib 은 글자마다 앞 폰트부터 찾고, 없으면 다음 폰트에서 가져온다. 맑은 고딕·나눔고딕에는 수학용 마이너스(U+2212)·근사 기호(U+2248)·곱셈 기호(×) 같은 기호가 없어 □ 로 깨졌는데, DejaVu Sans 가 이 기호들을 채운다.
- **axes.unicode_minus = False**: 축 눈금 숫자의 음수 기호만 일반 하이픈(-)으로 바꾼다. 제목·범례 문자열에는 효과가 없으므로 위의 대체 폰트가 필요하다.
- **순서가 중요**: sns.set_style() 이 matplotlib 글꼴 설정을 덮어쓸 수 있으므로 스타일을 먼저 적용하고 폰트를 나중에 지정한다.
- 마지막 print 로 실제 적용된 폰트 목록을 확인한다.

[출력 확인]
- 에러 없이 끝나고 **그래프 폰트: ['Malgun Gothic', 'DejaVu Sans']** 가 찍히면 정상 (Windows 기준. 다른 환경에서는 첫 항목이 그 환경의 한글 폰트).
- **ModuleNotFoundError: src** 가 나면 노트북을 injection_molding 폴더에서 연 것이 아니다.

[수정 포인트]
- 출력이 **['DejaVu Sans']** 하나뿐이면 한글 폰트가 없는 환경이다 (그래프 한글이 □). KAMP-NOTE 라면 %pip install --user koreanize-matplotlib 후 커널 재시작, 이 셀 끝에 import koreanize_matplotlib 를 추가한다.
- 다른 한글 폰트를 쓰고 싶으면 후보 목록 맨 앞에 폰트 이름을 넣는다.
- import platform 은 이전 방식(운영체제 감지)의 흔적으로 지금은 쓰지 않으므로 지워도 된다.

[셀 2] 0-2. 파이프라인 결과가 없으면 먼저 실행

목적 노트북이 읽는 결과 파일 12개(REQUIRED) 중 하나라도 없으면 run_all.main() 으로 전체 파이프라인을 돌린다.

```
# 결과 파일이 없으면 전체 파이프라인을 먼저 실행 (CPU 약 75분)
REQUIRED = ["cv_folds.csv", "oof_dev.csv", "test_predictions.csv", "leakage_cv.csv",
            "unlabeled_domain_auc.csv", "self_training_cv.csv", "regime_clusters.csv", "autoencoder_cv.csv",
            "significance.csv", "imbalance_cv.csv", "dl_stacking_cv.csv", "time_block_cv.csv"]
if not all((OUT_DIR / f).exists() for f in REQUIRED):
    import run_all
    run_all.main()
```

- REQUIRED: CV 결과·OOF·테스트 예측 + 부가 실험 결과(누수 비교, 비라벨 진단, 자기학습, 운전구간 군집, 오토인코더, 유의성 검정, 불균형 비교, 스태킹·딥러닝 비교, 시간 구간 검증).
- 모델 14개 조합 CV 와 부가 실험을 합쳐 CPU 로 약 75분 걸린다. 노트북에서 매번 돌리지 않도록 결과를 파일로 저장해 두고 재사용하는 구조다.
- 모든 파일이 있으면 아무 일도 하지 않는다.

[출력 확인]
- 처음 실행 시 'done LR_baseline ...' 6줄과 [6-1]~[6-5] 부가 실험 로그가 나온다.

[수정 포인트]
- 모델·CV 설정을 바꿨다면 **outputs/ 의 해당 csv 를 지우고** 이 셀을 다시 실행해야 새 결과가 반영된다.
- 부가 실험 없이 핵심 결과만 빠르게 보려면 src/config.py 의 RUN_EXTRA = False 로 두고 python run_all.py 를 실행한다 (단 이 경우 노트북 1-6-2, 2-7, 2-8, 3-8 셀은 파일이 없어 에러).

1장. 데이터 이해 및 진단 (보고서 1장 · 15점)

[셀 3] 1-0. 데이터 적재

목적 CN7·RG3 라벨 데이터를 읽어 한 표로 합치고, 사출(shot) 구조 컬럼을 붙인다.

```
df = load_all()
FEATS = process_cols(load_labeled("cn7"))
print("행 수:", len(df), "/ 공정변수:", len(FEATS))
df.head()
```

- load_all() 이 추가하는 컬럼: **shot_id**(같은 사출 번호), **cavity**(사출 내 순서 0/1), **shot_size**(사출당 제품 수), **product**(cn7/rg3), **group**(제품+사출번호, CV 그룹용).
- FEATS = 원본 공정변수 24개 목록. 이후 분석 대부분이 이 목록을 기준으로 한다.

[출력 확인]
- 행 수 2393, 공정변수 24개가 찍히면 정상.

[셀 4] 1-1. 데이터 진단표

목적 보고서 1장에 그대로 옮길 제품별 요약표를 만든다.

```
diag = diagnose(df)
diag.T
```

- fail_rate: 불량률. constant_cols: 값이 하나뿐이라 쓸모없는 변수(Clamp_Open_Position).
- shots / shots_2cavity: 사출 수와 그중 제품 2개가 나온 사출 수 → 2캐비티 구조의 근거.
- fail_at_cavity0: 첫 번째 캐비티에서 나온 불량 수 → 캐비티 위치가 중요하다는 근거.
- conflicting_patterns / fails_in_conflicting: 공정값이 완전히 같은데 양품·불량이 섞인 패턴 수와 그 안의 불량 수.

[출력 확인]
- CN7 불량 17 (1.40%), RG3 불량 25 (2.12%).
- 첫 캐비티 불량: CN7 13/17, RG3 23/25.
- 충돌 패턴 내 불량: CN7 11/17, RG3 25/25.

[수정 포인트]
- 보고서용으로 컬럼명을 한글로 바꾸려면 diag.rename(columns={...}) 를 덧붙인다.

[셀 5] 1-2. 클래스 불균형 그래프

목적 양품/불량 개수와 불량률을 막대그래프로 보여준다.

```
fig, ax = plt.subplots(1, 2, figsize=(10, 3.5))
cnt = df.groupby(["product", TARGET]).size().unstack()
cnt.plot.bar(ax=ax[0], rot=0, color=["#9bb7d4", "#d9534f"])
ax[0].set_title("제품별 양품(0) / 불량(1) 수")
(df.groupby("product")[TARGET].mean() * 100).plot.bar(ax=ax[1], rot=0, color="#d9534f")
ax[1].set_title("제품별 불량률 (%)")
plt.tight_layout(); plt.show()
```

- 왼쪽: 제품별 양품(0)·불량(1) 수. 오른쪽: 불량률(%).
- 불량이 1~2% 뿐이라 '전부 양품'이라고 찍어도 정확도 98% 가 나온다 → 정확도(Accuracy) 대신 PR-AUC·F1 을 써야 하는 이유.

[출력 확인]
- 불량 막대가 거의 보이지 않을 정도로 작아야 정상.

[수정 포인트]
- 색상은 color=[양품색, 불량색] 리스트에서 바꾼다.

[셀 6] 1-3. 2캐비티 사출 구조 표

목적 사출 단위 통계와 캐비티(0/1)별 불량률을 표로 확인한다.

```
shot_tbl = (df.groupby("product")
              .apply(lambda d: pd.Series({
                  "사출 수": d.shot_id.nunique(),
                  "2개 제품 사출": (d.groupby("shot_id").size() == 2).sum(),
                  "1개 제품 사출": (d.groupby("shot_id").size() == 1).sum(),
                  "불량 사출": (d.groupby("shot_id")[TARGET].sum() > 0).sum(),
              })))
display(shot_tbl)

cav = pd.crosstab([df["product"], df["cavity"]], df[TARGET], margins=False)
cav["불량률(%)"] = (cav[1] / cav.sum(axis=1) * 100).round(2)
display(cav)
```

- shot_tbl: 제품별 사출 수, 2개/1개 제품 사출 수, 불량이 하나라도 있는 사출 수.
- pd.crosstab: (제품, 캐비티) × (양품, 불량) 교차표. 불량률(%) 컬럼을 추가로 계산한다.
- **왜 중요한가**: 한 사출의 두 제품은 공정값이 완전히 같다. 둘이 학습/검증에 나뉘면 모델이 '정답을 본' 셈이 되어 성능이 부풀려진다 → 사출 단위 그룹 CV 를 쓰는 근거.

[출력 확인]
- 캐비티 0 의 불량률이 캐비티 1 보다 확연히 높아야 한다.

[셀 7] 1-4. 캐비티별 불량률 그래프

목적 1-3 의 캐비티별 불량률을 막대그래프로 시각화 (보고서 삽입용).

```
fig, ax = plt.subplots(figsize=(6, 3.5))
(df.groupby(["product", "cavity"])[TARGET].mean() * 100).unstack().plot.bar(ax=ax, rot=0)
ax.set_title("캐비티 위치별 불량률 (%)"); ax.set_ylabel("%")
plt.tight_layout(); plt.show()
```

[출력 확인]
- cavity 0 막대가 높게 나온다.

[수정 포인트]
- 보고서용으로 저장하려면 plt.show() 앞에 plt.savefig('outputs/fig_cavity.png', dpi=200) 추가.

[셀 8] 1-5. 라벨 충돌 확인

목적 공정값 패턴이 같은 행끼리 묶어 양품·불량이 섞인 경우를 센다.

```
rows = []
for p, d in df.groupby("product"):
    pat = d.groupby(FEATS)[TARGET].agg(["size", "nunique", "sum"])
    rows.append({"product": p,
                 "불량 수": int(d[TARGET].sum()),
                 "충돌 패턴 내 불량": int(pat.loc[pat["nunique"] > 1, "sum"].sum()),
                 "순수 불량 패턴": int((pat["sum"] == pat["size"]).sum())})
pd.DataFrame(rows)
```

- groupby(FEATS): 24개 공정변수 값이 모두 같은 행끼리 묶는다.
- nunique > 1 이면 같은 공정값에 양품과 불량이 모두 있다는 뜻 → 공정변수만으로는 원리적으로 구분 불가.
- **보고서 메시지**: 'RG3 불량은 전부 같은 공정값의 양품 짝이 존재 → 공정 데이터만으로는 한계, 캐비티(금형) 정보가 필요'.

[출력 확인]
- RG3: 불량 25, 충돌 패턴 내 불량 25.

[셀 9] 1-6. 결측·상수·스케일 점검

목적 결측치, 상수 컬럼, 라벨/비라벨 데이터의 표준화 상태를 확인한다.

```
print("결측치 합계:", int(df[FEATS].isna().sum().sum()))
print("상수 컬럼:", [c for c in FEATS if df[c].nunique() == 1])
scaling_check()   # 라벨/비라벨 데이터가 각자 따로 표준화됨 → 결합하지 않음
```

- scaling_check(): 각 파일의 평균·표준편차를 보여준다. 라벨 데이터도 비라벨 데이터도 각자 평균 0·표준편차 1 → **각 파일을 따로 표준화**했다는 증거.
- 따로 표준화된 데이터는 같은 값이 같은 실제 값을 뜻하지 않는다. 그래서 바로 다음 셀(1-6-2)에서 비라벨 데이터를 섞어도 되는지 실험으로 확인한다.

[출력 확인]
- 결측 0, 상수 컬럼 ['Clamp_Open_Position'], max_abs_mean 약 0.

[셀 10] 1-6-2. 비라벨 데이터 결합 가능성 진단 (도메인 판별 · 자기학습)

목적 비라벨 7만 행을 학습에 써도 되는지 두 가지 실험 결과(run_all 이 저장)를 불러와 확인한다.

```
display(pd.read_csv(OUT_DIR / "unlabeled_domain_auc.csv").round(3))

st = pd.read_csv(OUT_DIR / "self_training_cv.csv")
cols = ["PR_AUC_all", "PR_AUC_cn7", "PR_AUC_rg3", "Recall@20%"]
display(st.groupby("method")[cols].agg(["mean", "std"]).round(3))
print("비라벨 평균 예측 불량확률: %.3f  (라벨 데이터 불량률 %.3f)"
      % (st["unlabeled_mean_prob"].mean(), df[TARGET].mean()))
```

- **도메인 판별** (unlabeled_domain_auc.csv): 라벨 행(0)과 비라벨 행(1)을 맞히는 RandomForest 의 CV AUC. 0.5 면 두 데이터가 같은 분포라 구분 불가, 1.0 이면 완전히 다른 분포. rank 는 파일별 순위 변환으로 표준화 방식 차이를 없앤 결과.
- **자기학습** (self_training_cv.csv): 라벨로 학습한 RF 가 비라벨에 매긴 확률 상위(strict 0.5% / loose 2%)를 불량, 하위(50% / 80%, 최대 5,000행)를 양품 가짜 라벨로 붙여 학습 데이터에 추가하고, 같은 그룹 CV fold 에서 라벨만 쓴 모델(labeled_only)과 비교한다.
- **읽는 법**: 효과가 제품·지표마다 같은 방향이면 '도움이 된다', 엇갈리면 '불안정하다'. 비라벨 평균 예측 불량확률이 실제 불량률과 크게 다르면 모델이 처음 보는 분포에서 헛짚고 있다는 신호.
- **보고서 메시지**: '분포가 완전히 다르고(AUC 1.00) 자기학습 효과도 불안정해 비라벨 데이터는 학습에 결합하지 않았다' → 결합하지 않은 이유를 추측이 아닌 실험으로 제시.

[출력 확인]
- 도메인 AUC: CN7·RG3, raw·rank 모두 1.0.
- 자기학습: PR-AUC 전체 0.124 → 0.140(strict)·0.160(loose)으로 오르나 RG3 는 0.106 → 0.083·0.088 로 하락, Recall@20% 도 0.612 → 0.593·0.529 로 하락.
- 비라벨 평균 예측 불량확률 0.039 (실제 불량률 0.018 의 약 2배).

[수정 포인트]
- 가짜 라벨 비율은 src/experiments.py 의 self_training() 안 settings 딕셔너리, 반복 횟수는 config.py 의 SEMI_REPEATS 로 조정.

[셀 11] 1-7. 생산 순번별 불량 분포

목적 원본 순번(Unnamed: 0)을 x축으로 불량 발생 위치를 찍는다.

```
fig, axes = plt.subplots(2, 1, figsize=(12, 4), sharey=True)
for ax, (p, d) in zip(axes, df.groupby("product")):
    ax.scatter(d[INDEX_COL], d[TARGET], s=4, alpha=.6, c=d[TARGET], cmap="coolwarm")
    ax.set_title(f"{p.upper()} - 생산 순번별 불량 발생"); ax.set_yticks([0, 1])
plt.tight_layout(); plt.show()
```

- CN7 불량은 순번 47~120 구간에 몰려 있다 → 특정 시기에 일어난 '사건'일 가능성.
- RG3 불량은 전 구간에 흩어져 있다.

[출력 확인]
- 위쪽 줄(1)에 찍힌 점의 위치를 본다.

[수정 포인트]
- 산점도가 너무 빽빽하면 s(점 크기)와 alpha(투명도)를 조정.

[셀 12] 1-8. 양품 vs 불량 평균 차이 히트맵

목적 변수별로 '불량 평균 - 양품 평균'을 계산해 어떤 공정변수가 불량과 관련 있는지 한눈에 본다.

```
use = [c for c in FEATS if df[c].nunique() > 1]
diff = (df.groupby(["product", TARGET])[use].mean()
          .groupby(level=0).diff().dropna().droplevel(1))
fig, ax = plt.subplots(figsize=(12, 3))
sns.heatmap(diff, cmap="RdBu_r", center=0, annot=True, fmt=".1f", ax=ax, cbar=False)
ax.set_title("불량 평균 - 양품 평균 (표준화 단위)")
plt.tight_layout(); plt.show()
```

- use = 상수 컬럼을 뺀 변수 목록.
- groupby([product, TARGET]).mean() 후 diff() → 불량 - 양품. 값이 표준화 단위라 ±1 이면 표준편차 1개만큼 차이.
- 빨강: 불량일 때 더 큼, 파랑: 불량일 때 더 작음.

[출력 확인]
- 절댓값이 큰 칸이 후보 영향변수.

[셀 13] 1-9. 상위 차이 변수 박스플롯

목적 1-8 에서 차이가 큰 변수 6개를 골라 양품/불량 분포를 비교한다.

```
top = diff.abs().max().sort_values(ascending=False).index[:6]
fig, axes = plt.subplots(2, 3, figsize=(13, 6))
for ax, c in zip(axes.ravel(), top):
    sns.boxplot(data=df, x="product", y=c, hue=TARGET, ax=ax, fliersize=2)
    ax.set_title(c); ax.legend_.remove()
plt.tight_layout(); plt.show()
```

- top = 차이 절댓값 기준 상위 6개. 개수를 바꾸려면 [:6] 과 subplots(2, 3) 을 함께 바꾼다.

[출력 확인]
- 불량(주황) 박스가 양품과 확연히 다른 위치에 있는 변수를 메모.

[수정 포인트]
- [:6] → [:9], subplots(2, 3) → subplots(3, 3) 처럼 짝을 맞춘다.

[셀 14] 1-10. 공정변수 상관 히트맵

목적 변수 간 상관을 제품별로 보여준다. 거의 같은 정보를 담은 변수 묶음을 찾는 용도.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
for ax, (p, d) in zip(axes, df.groupby("product")):
    sns.heatmap(d[use].corr(), cmap="RdBu_r", center=0, ax=ax, cbar=False)
    ax.set_title(f"{p.upper()} 공정변수 상관")
plt.tight_layout(); plt.show()
```

- 예: Barrel_Temperature_1~6 끼리, 시간 관련 변수끼리 상관이 높게 나오면 SHAP 해석 시 '묶음'으로 해석해야 한다.

[수정 포인트]
- 숫자를 보고 싶으면 annot=True, fmt='.1f' 추가 (칸이 많아 글씨가 작아짐).

2장. AI 예측모델 개발 및 성능평가 (보고서 2장 · 40점)

[셀 15] 2-1. CV 결과 요약

목적 run_all 이 저장한 fold 별 성능(cv_folds.csv)을 읽어 모델·특성셋별 평균±표준편차를 만든다.

```
cv = pd.read_csv(OUT_DIR / "cv_folds.csv")
MET = ["PR_AUC", "ROC_AUC", "F1", "Precision", "Recall", "Recall@5%", "Recall@10%", "Recall@20%"]
summary = cv.groupby(["scope", "features", "model"])[MET].agg(["mean", "std"]).round(3)
summary.loc["all"]
```

- scope: all(두 제품 합쳐서), cn7, rg3.
- features: process_only(공정변수만) / process+cavity(캐비티 추가).
- PR_AUC: 불량을 위로 얼마나 잘 줄세우는지(임계값 무관, 주지표). F1: 임계값 적용 후 정밀도·재현율 조화평균. Recall@k%: 상위 k% 만 검사할 때 잡는 불량 비율.

[출력 확인]
- 현재 결과: ExtraTrees + 캐비티가 PR-AUC 0.139 로 최고 → 최종모델. RF+ET 앙상블 0.130, RandomForest 0.114, LR 베이스라인 0.109 (공정변수만 0.088). 부스팅(XGBoost 0.099, CatBoost 0.098)은 배깅보다 낮다.

[수정 포인트]
- 표 전체를 보려면 summary (loc 없이) 를 출력. 제품별은 summary.loc['cn7'].

[셀 16] 2-2. PR-AUC 분포 박스플롯

목적 25개 fold 의 PR-AUC 흩어짐을 모델·특성셋·제품별로 비교한다.

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=False)
for ax, sc in zip(axes, ["all", "cn7", "rg3"]):
    sns.boxplot(data=cv[cv.scope == sc], x="model", y="PR_AUC", hue="features", ax=ax)
    ax.set_title(f"PR-AUC 분포 ({sc})"); ax.set_xlabel("")
plt.tight_layout(); plt.show()
```

- 평균만 보면 우연한 차이인지 알 수 없다. 박스가 겹치면 차이가 불확실하다는 뜻 → 보고서에 '분산이 크다'는 점을 솔직히 적는 근거.

[출력 확인]
- RG3 패널에서 process_only 박스가 바닥 근처인지 확인.

[셀 17] 2-3. 평균 성능 막대 + 무작위 기준선

목적 PR-AUC, F1, Recall@20% 를 막대로 그리고, 무작위 모델의 PR-AUC(= 불량률 약 0.018)를 점선으로 표시.

```
base = df[TARGET].mean()
tbl = cv[cv.scope == "all"].groupby(["model", "features"])[["PR_AUC", "F1", "Recall@20%"]].mean().unstack()
tbl.plot.bar(figsize=(12, 4), rot=0)
plt.axhline(base, ls="--", c="gray", label=f"무작위 PR-AUC ≈ {base:.3f}")
plt.title("모델 × 특성셋 평균 성능 (전체)"); plt.legend(bbox_to_anchor=(1, 1)); plt.tight_layout(); plt.show()
```

- PR-AUC 는 무작위일 때 불량률과 같아진다. 점선보다 몇 배 높은지가 모델의 실제 가치.

[셀 18] 2-4. 최종모델 선정 및 테스트 성능

목적 CV PR-AUC 최고 조합을 고르고, 개발셋 전체로 다시 학습해 테스트셋에서 평가한다.

```
allscope = cv[cv.scope == "all"].groupby(["model", "features"])["PR_AUC"].mean()
BEST_MODEL, BEST_FS = allscope.idxmax()
USE_CAV = BEST_FS == "process+cavity"
print(f"최종모델: {BEST_MODEL} / {BEST_FS}  (CV PR-AUC {allscope.max():.3f})")

dev, test = holdout_split(df)
X_dev, X_test = feature_matrix(dev, USE_CAV), feature_matrix(test, USE_CAV)
y_dev, y_test = dev[TARGET].to_numpy(), test[TARGET].to_numpy()

model = get_models()[BEST_MODEL]
THR = best_threshold(y_dev, inner_oof(model, X_dev, y_dev, dev["group"].to_numpy(), SEED))
final = clone(model).fit(X_dev, y_dev)
p_test = final.predict_proba(X_test)[:, 1]

res = {"all": metrics(y_test, p_test, THR)}
for p in ["cn7", "rg3"]:
    m = test["product"].to_numpy() == p
    res[p] = metrics(y_test[m], p_test[m], THR)
print("임계값:", round(THR, 4))
pd.DataFrame(res).round(3)
```

- holdout_split: run_all.py 와 동일한 방식(사출 단위·불량 층화, 20%)으로 테스트셋을 나눈다.
- THR(임계값): 개발셋 내부 3-fold 예측으로만 F1 최대 지점을 고른다. 테스트셋은 임계값 선정에 쓰지 않는다(정보 누수 방지).
- final: 개발셋 전체로 학습한 최종모델. p_test: 테스트 불량확률.

[출력 확인]
- 테스트 불량이 8건뿐이라 수치 변동이 크다. 보고서에서는 CV 결과를 주 근거로, 테스트는 '보조 확인'으로 쓴다.

[수정 포인트]
- 특정 모델을 강제로 쓰려면 BEST_MODEL, BEST_FS = 'LightGBM', 'process+cavity' 처럼 직접 지정.

[셀 19] 2-5. 혼동행렬 · PR 곡선

목적 테스트셋에서 맞힌/틀린 개수와 제품별 PR 곡선을 그린다.

```
from sklearn.metrics import ConfusionMatrixDisplay, PrecisionRecallDisplay

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ConfusionMatrixDisplay.from_predictions(y_test, (p_test >= THR).astype(int), ax=ax[0],
                                        display_labels=["양품", "불량"], colorbar=False)
ax[0].set_title("테스트 혼동행렬")
for p in ["cn7", "rg3"]:
    m = test["product"].to_numpy() == p
    PrecisionRecallDisplay.from_predictions(y_test[m], p_test[m], ax=ax[1], name=p.upper())
ax[1].set_title("테스트 PR 곡선")
plt.tight_layout(); plt.show()
```

- 혼동행렬: 왼쪽 위 = 양품을 양품으로(TN), 오른쪽 아래 = 불량을 불량으로(TP), 오른쪽 위 = 오경보(FP), 왼쪽 아래 = 미탐지(FN).
- PR 곡선: 오른쪽 위에 가까울수록 좋다. 곡선 아래 면적 = PR-AUC(AP).

[셀 20] 2-6. 확률 보정 곡선

목적 모델이 말하는 '불량확률 30%' 가 실제로 30% 인지 점검한다.

```
from sklearn.calibration import calibration_curve

oof = pd.read_csv(OUT_DIR / "oof_dev.csv")
frac, mean_p = calibration_curve(oof[TARGET], oof["oof_prob"], n_bins=8, strategy="quantile")
plt.figure(figsize=(4.5, 4))
plt.plot(mean_p, frac, "o-", label=BEST_MODEL)
plt.plot([0, mean_p.max()], [0, mean_p.max()], "--", c="gray", label="완전 보정")
plt.xlabel("예측 불량확률"); plt.ylabel("실제 불량비율"); plt.title("보정 곡선 (dev OOF)")
plt.legend(); plt.tight_layout(); plt.show()
```

- 점이 점선(완전 보정) 위에 있으면 확률을 그대로 믿을 수 있다. 벗어나면 확률 보정(Platt/Isotonic)이 필요 → 창의성 항목의 '확률보정' 재료.
- quantile 구간 8개로 나눠 각 구간의 평균 예측확률(x)과 실제 불량비율(y)을 비교.

[수정 포인트]
- 보정을 적용하려면 sklearn.calibration.CalibratedClassifierCV(model, method='sigmoid', cv=3) 로 감싸 비교해 본다.

[셀 21] 2-7. 검증 방식에 따른 성능 과대평가 (누수 비교)

목적 같은 모델·같은 반복 횟수로 행 단위 CV 와 사출 단위 그룹 CV 의 PR-AUC 를 비교해, 그룹 CV 를 쓴 이유를 숫자로 보여준다.

```
lk = pd.read_csv(OUT_DIR / "leakage_cv.csv")
tbl = lk.groupby(["model", "scheme"])["PR_AUC"].agg(["mean", "std"]).unstack("scheme")
tbl[("over", "과대평가(%)")] = ((tbl[("mean", "row")] / tbl[("mean", "group")] - 1) * 100).round(1)
display(tbl.round(3))

ax = sns.barplot(data=lk, x="model", y="PR_AUC", hue="scheme", hue_order=["row", "group"], errorbar="sd")
ax.set_title("검증 방식별 PR-AUC (row = 행 단위, group = 사출 단위)")
plt.tight_layout(); plt.show()
```

- **row**: 일반적인 StratifiedKFold. 같은 사출의 두 제품이 학습·검증에 갈라질 수 있다 (누수). **group**: 본 프로젝트의 StratifiedGroupKFold.
- 과대평가(%) = (row 평균 ÷ group 평균 - 1) × 100. 행 단위로 검증했다면 성능을 이만큼 부풀려 보고했을 것이라는 뜻.
- 막대 위 검은 선은 표준편차. 막대가 겹쳐 보여도 같은 fold 구성에서 25번 반복한 평균 차이이므로 방향은 일관된다.
- **보고서 메시지**: 5장 창의성의 핵심 근거. '데이터 구조를 무시한 검증은 최종모델 성능을 43% 부풀린다'. 성능이 좋은 모델일수록 과대평가 폭이 크다.

[출력 확인]
- LR: row 0.143 vs group 0.109 (+32%). RandomForest: 0.155 vs 0.114 (+36%). ExtraTrees: 0.198 vs 0.139 (+43%).

[셀 22] 2-8. 비지도 이상탐지(오토인코더)와 비교

목적 양품만으로 학습한 오토인코더의 복원 오차를 불량 점수로 써서, 지도학습 모델과 같은 fold 에서 비교한다.

```
ae = pd.read_csv(OUT_DIR / "autoencoder_cv.csv")
sup = cv[(cv.features == "process+cavity")].pivot_table(index=["model", "repeat", "fold"], columns="scope",
                                                        values="PR_AUC").reset_index()
comp = pd.concat([
    sup.groupby("model")[["all", "cn7", "rg3"]].mean(),
    ae[["PR_AUC_all", "PR_AUC_cn7", "PR_AUC_rg3"]].mean().set_axis(["all", "cn7", "rg3"]).to_frame("AutoEncoder
    (비지도)").T,
])
comp.loc["무작위 기준"] = [df[TARGET].mean(), df.loc[df["product"] == "cn7", TARGET].mean(),
                          df.loc[df["product"] == "rg3", TARGET].mean()]
comp.round(3).rename(columns={"all": "PR-AUC 전체", "cn7": "CN7", "rg3": "RG3"})
```

- **오토인코더**: 입력을 작은 차원(6개)으로 압축했다가 다시 복원하도록 학습한 신경망 (sklearn MLPRegressor, 은닉층 16-6-16). 양품만 보고 학습했으므로 양품은 잘 복원하고, 평소와 다른 공정값은 복원 오차가 커진다.
- 라벨(불량 여부)을 학습에 쓰지 않는 **비지도 이상탐지**라 라벨이 부족한 현장에서 많이 쓰인다. 우수사례(도어핸들 사출)도 이 방식을 사용했다.
- comp 표: 지도학습 7종은 +캐비티 특성셋의 fold 평균, 오토인코더는 같은 그룹 CV 평균, 마지막 줄은 무작위 기준(= 불량률).
- **해석 주의**: 통합 PR-AUC 는 오토인코더가 약간 높지만 RG3 는 무작위 수준이고 Recall@20%(autoencoder_cv.csv) 는 최종모델의 40% 수준이다. 그래서 검사 우선순위는 ExtraTrees, 오토인코더는 가동 초기 같은 '공정 이상 경보' 보조로 역할을 나눈다.

[출력 확인]
- AutoEncoder: PR-AUC 전체 0.123 / CN7 0.274 / RG3 0.026, Recall@20% 0.281 (최종모델 ExtraTrees 0.707).
- 무작위 기준: 전체 0.018 / CN7 0.014 / RG3 0.021.

[수정 포인트]
- 은닉층 구조·학습 횟수는 src/experiments.py 의 autoencoder_cv() 안 MLPRegressor 인자로 조정.

[셀 23] 2-9. 최종모델 대비 유의성 검정

목적 최종모델과 나머지 13개 조합의 차이가 우연인지 같은 fold 짝 비교로 검정한다 (significance.csv).

```
sig = pd.read_csv(OUT_DIR / "significance.csv")
print("최종모델:", BEST_MODEL, "/", BEST_FS)
for met in ["PR_AUC", "Recall@20%"]:
    print(f"Wn■ {met}")
    display(sig[sig.metric == met][["compared_model", "features", "best_mean", "other_mean", "mean_diff",
                                    "wins", "losses", "wilcoxon_p", "corrected_t_p"]].round(3))
```

- **wins / losses**: 25개 fold 중 최종모델이 이긴 / 진 횟수. 평균만 보는 것보다 차이의 일관성을 보여준다.
- **Wilcoxon 부호순위 검정**: 짝지어진 차이의 부호와 크기 순위로 판단하는 비모수 검정. 반복 CV 의 fold 들은 학습 데이터가 겹쳐 서로 독립이 아니므로 다소 낙관적이다.
- **Nadeau-Bengio 보정 t-검정**: 반복 k-fold 의 겹침을 분산에 반영(1/k + 검증/학습 비율)한 보수적 검정. 두 검정이 모두 유의하면 강한 근거, 하나만 유의하면 '경향'으로 쓴다.
- **보고서 메시지**: '모델 간 PR-AUC 차이는 대부분 유의하지 않지만 캐비티 특성의 효과는 보수적 검정에서도 유의 → 성능을 결정한 것은 데이터 구조'.

[출력 확인]
- vs RandomForest(+캐비티): PR-AUC 15승 10패, Wilcoxon p=0.30 / Recall@20% 14승 2패, Wilcoxon p=0.008, 보정 t p=0.21.
- vs ExtraTrees(공정변수만): PR-AUC 23승 2패, Wilcoxon p<0.001 / Recall@20% 보정 t p=0.011.

[수정 포인트]
- 비교 지표는 src/experiments.py 의 significance_tests(metrics=(...)) 로 바꾼다.

[셀 24] 2-10. 불균형 처리 방식 비교

목적 최종모델 구조는 그대로 두고 불균형 처리만 바꿔 같은 25개 fold 에서 비교한다 (imbalance_cv.csv).

```
imb = pd.read_csv(OUT_DIR / "imbalance_cv.csv")
cols = ["PR_AUC_all", "PR_AUC_cn7", "PR_AUC_rg3", "Recall@20%"]
tbl = imb.groupby("method")[cols].agg(["mean", "std"]).round(3)
display(tbl.sort_values(("PR_AUC_all", "mean"), ascending=False))

ax = sns.boxplot(data=imb, x="method", y="PR_AUC_all", order=tbl.sort_values(("PR_AUC_all", "mean"), ascending=F
    alse).index)
ax.set_title("불균형 처리 방식별 PR-AUC (25 fold)"); ax.set_xlabel("")
plt.xticks(rotation=15); plt.tight_layout(); plt.show()
```

- **클래스 가중치**: 불량 1건의 오분류를 더 무겁게 계산 (현재 방식). **가중치 없음**: 비교 기준.
- **SMOTE-NC**: 불량 표본 사이를 보간해 합성 불량을 만들되 범주형(is_rg3, cavity)은 보간하지 않는다. **BorderlineSMOTE**: 양품과 가까운 경계의 불량 주변에 집중. **ADASYN**: 학습이 어려운 불량 주변에 더 많이 생성.
- 오버샘플링은 **학습 fold 에만** 적용하고(imblearn 파이프라인), 이때는 클래스 가중치를 꺼서 이중 보정을 막는다.
- **해석**: 공정값이 같은 양품·불량 짝이 많은 데이터에서는 합성 표본이 구분 불가능한 영역의 '가짜 불량'을 늘려 오히려 성능을 낮춘다.

[출력 확인]
- PR-AUC: 가중치 없음 0.144, 클래스 가중치 0.139, BorderlineSMOTE 0.125, ADASYN 0.118, SMOTE-NC 0.114.
- Recall@20%: 클래스 가중치 0.707, ADASYN 0.712, SMOTE-NC 0.691, 가중치 없음 0.686.

[수정 포인트]
- 비교 방식은 src/experiments.py 의 imbalance_comparison() 안 variants 딕셔너리에 추가한다.

[셀 25] 2-11. 스태킹 앙상블 · 딥러닝(MLP) 검토

목적 선행연구가 쓴 스태킹 앙상블과 표 데이터용 딥러닝(MLP)을 최종모델과 같은 25개 fold 에서 비교한다 (dl_stacking_cv.csv).

```
dl = pd.read_csv(OUT_DIR / "dl_stacking_cv.csv")
cols = ["PR_AUC_all", "ROC_AUC", "Recall@20%", "PR_AUC_cn7", "PR_AUC_rg3"]
tbl = dl.groupby("model", sort=False)[cols].mean()
ref = dl[dl.model == "최종모델"].set_index(["repeat", "fold"])["PR_AUC_all"]
tbl["최종모델 대비 승/패"] = [
    "-" if m == "최종모델" else
    f"{int((d.set_index(['repeat', 'fold'])['PR_AUC_all'] > ref).sum())}/{int((d.set_index(['repeat', 'fold'])['
    PR_AUC_all'] < ref).sum())}"
    for m, d in dl.groupby("model", sort=False)]
display(tbl.round(3))

ax = sns.boxplot(data=dl, x="model", y="PR_AUC_all")
ax.axhline(df[TARGET].mean(), ls="--", c="gray", label="무작위 기준")
ax.set_title("최종모델 vs 스태킹 · 딥러닝(MLP) PR-AUC (25 fold)"); ax.set_xlabel(""); ax.legend()
plt.xticks(rotation=10); plt.tight_layout(); plt.show()
```

- **스태킹**: ExtraTrees · RandomForest · XGBoost · LR 의 예측 확률을 입력으로 로지스틱 회귀(메타 모델)가 최종 확률을 낸다. 메타 모델 학습용 내부 CV 도 사출 단위로 묶어야 누수가 없어서, 학습 fold 의 그룹으로 분할을 미리 만들어 cv 인자로 넘긴다.
- **MLP**: 은닉층이 있는 신경망(딥러닝의 기본형). sklearn MLP 는 class_weight 가 없어 학습 fold 안에서만 불량을 무작위 복제(RandomOverSampler)하고, early_stopping 으로 과적합을 줄인다.
- tbl 의 '최종모델 대비 승/패': 같은 fold 짝 비교로 25번 중 몇 번 이겼는지.
- **CNN · LSTM 을 넣지 않은 이유**: 이미지(공간 구조) · 연속 시계열(시간 구조)용 구조라, 사출 1회당 공정값 24개인 표 데이터에는 맞지 않는다.
- **보고서 메시지**: '더 복잡한 앙상블과 딥러닝은 불량 42건 규모에서 이득이 없다 → 데이터 규모와 형태에 맞는 배깅 앙상블을 유지'.

[출력 확인]
- 최종모델(ExtraTrees): PR-AUC 0.139 / Recall@20% 0.707.
- 스태킹: 0.127 / 0.560 (13승 12패). MLP 32-16: 0.091 / 0.473 (6/19). MLP 64-32-16: 0.099 / 0.493 (3/22).

[수정 포인트]
- 후보는 src/experiments.py 의 dl_stacking_comparison() 안 fixed 딕셔너리, 스태킹 구성은 _stacking() 의 estimators 에서 바꾼다.

[셀 26] 2-12. 시간 구간 검증 — 피드백 운영 vs 처음 보는 생산 구간

목적 연속 생산 구간을 통째로 평가용으로 빼는 검증으로, 처음 보는 불안정 구간에서의 성능과 시간 누수 정도를 확인한다 (time_block_cv.csv).

```
tb = pd.read_csv(OUT_DIR / "time_block_cv.csv")
cols = ["PR_AUC_all", "PR_AUC_cn7", "PR_AUC_rg3", "ROC_AUC", "Recall@20%"]

# ① 두 시나리오: 최종모델의 성능 범위
shot = cv[(cv.model == BEST_MODEL) & (cv.features == BEST_FS)].pivot_table(
    index=["repeat", "fold"], columns="scope", values=["PR_AUC", "ROC_AUC", "Recall@20%"])
scen = pd.DataFrame({
    "피드백 운영 (사출 단위 CV)": [shot[("PR_AUC", "all")].mean(), shot[("PR_AUC", "cn7")].mean(), shot[("PR_AUC", "rg3")
    ].mean(),
                                shot[("ROC_AUC", "all")].mean(), shot[("Recall@20%", "all")].mean()],
    **{f"처음 보는 구간 (시간 구간 {nb}개)": tb[(tb.model == BEST_MODEL) & (tb.features == BEST_FS) & (tb.n_blocks == nb)][
    cols].mean().values
       for nb in sorted(tb.n_blocks.unique())}}, index=cols)
display(scen.round(3))

# ② 시간 구간 검증에서의 모델 순위와 캐비티 효과
rank = tb.groupby(["n_blocks", "features", "model"])[cols].mean().round(3)
display(rank.sort_values(["n_blocks", "PR_AUC_all"], ascending=[True, False]))
```

- **왜 필요한가**: CN7 불량은 생산 초반 한 구간에 몰려 있다. 사출 단위 CV 는 같은 시기의 다른 사출이 학습에 들어가므로 '그 시기 = 불량' 을 외울 수 있다. 이를 **시간 누수**라 한다.
- **시간 구간 그룹**: 제품별 생산순번을 연속 구간 8개·16개로 나눠(qcut) 구간 전체를 하나의 그룹으로 묶고, StratifiedGroupKFold(4-fold × 3회)로 나눈다. 평가 구간에 불량이 없는 fold 는 제외한다.
- **두 시나리오로 해석**: 사출 단위 CV = 검사 결과를 계속 라벨로 쌓아 재학습하는 '피드백 운영', 시간 구간 CV = 재학습 없이 '처음 보는 구간'.
- scen 표: 최종모델의 두 시나리오 성능. rank 표: 시간 구간 CV 에서의 모델 순위와 캐비티 효과(ExtraTrees process_only 행).
- **보고서 메시지**: '처음 보는 구간에서는 PR-AUC 약 3배 수준 → 주기적 재학습 필수. 캐비티 효과는 검증 방식과 무관하게 유지'.

[출력 확인]
- 피드백 운영: PR-AUC 0.139 / ROC 0.841 / Recall@20% 0.707.
- 처음 보는 구간: 구간 8개 0.048 / 0.724 / 0.435, 구간 16개 0.060 / 0.774 / 0.635.
- ET 공정변수만(시간 구간): 0.034 · 0.039 → 캐비티 효과 유지.

[수정 포인트]
- 구간 수는 src/experiments.py 의 time_block_validation(n_blocks=(8, 16)) 으로 바꾼다. 구간이 너무 적으면 fold 마다 불량이 없어 집계가 불안정하다.

3장. 영향요인 및 오류분석 (보고서 3장 · 15점)

[셀 27] 3-1. SHAP 요약 그래프

목적 최종모델이 불량확률을 올리거나 내릴 때 어떤 변수를 쓰는지 보여준다.

```
import shap

from src.experiments import shap_positive

sv1 = shap_positive(final, X_dev)   # 불량 클래스 SHAP (앙상블은 구성 모델 SHAP 의 평균)
shap.summary_plot(sv1, X_dev, max_display=15, show=False)
plt.title("SHAP — 불량확률 기여"); plt.tight_layout(); plt.show()
```

- 점 하나 = 제품 하나. x축 = 그 변수가 불량확률을 얼마나 밀어올렸나(+)/내렸나(-). 색 = 변수 값(빨강 높음, 파랑 낮음).
- sv1: src/experiments.py 의 shap_positive() 로 불량(1) 클래스 SHAP 을 계산한다. 트리 모델은 TreeExplainer 를 쓰고, RF+ET 같은 소프트 보팅 앙상블이 최종모델이 되면 구성 모델 SHAP 의 평균을 쓴다(확률 평균이므로 정확히 일치).

[출력 확인]
- cavity 와 is_rg3 가 상위에 나오는지 확인.

[수정 포인트]
- ExtraTrees 500그루라 수 분 걸릴 수 있다. 빠르게 보려면 X_dev.sample(500, random_state=0) 로 줄인다.

[셀 28] 3-2. 주요 변수 · 의존도 그래프

목적 SHAP 평균 절댓값으로 중요도 순위를 매기고, 상위 공정변수 2개(TOP2)의 의존도 그래프를 그린다.

```
imp = pd.Series(np.abs(sv1).mean(0), index=X_dev.columns).sort_values(ascending=False)
TOP2 = [c for c in imp.index if c not in ("is_rg3", "cavity")][:2]
print("상위 변수:", imp.head(10).round(4).to_dict())
for c in TOP2:
    shap.dependence_plot(c, sv1, X_dev, interaction_index="auto", show=False)
    plt.tight_layout(); plt.show()
```

- TOP2 는 cavity·is_rg3 를 제외한 공정변수 중 상위 2개. 3-3 상호작용 분석에 쓰인다.
- dependence_plot: x = 변수 값, y = SHAP. interaction_index='auto' 가 가장 강하게 얽힌 다른 변수로 색을 칠한다 → 상호작용 단서.

[출력 확인]
- 출력된 '상위 변수' 딕셔너리를 보고서 표로 옮긴다.

[수정 포인트]
- 다른 변수를 보고 싶으면 TOP2 = ['변수A', '변수B'] 로 직접 지정.

[셀 29] 3-3. 상위 2개 변수 구간별 불량률

목적 TOP2 두 변수를 각각 4구간(사분위)으로 나눠 조합별 실제 불량률 히트맵을 만든다.

```
tmp = dev.copy()
for c in TOP2:
    tmp[c + "_bin"] = pd.qcut(tmp[c], 4, duplicates="drop")
pv = tmp.pivot_table(index=TOP2[0] + "_bin", columns=TOP2[1] + "_bin", values=TARGET, aggfunc="mean") * 100
plt.figure(figsize=(7, 4.5))
sns.heatmap(pv, annot=True, fmt=".1f", cmap="Reds")
plt.title(f"{TOP2[0]} × {TOP2[1]} 구간별 불량률 (%)"); plt.tight_layout(); plt.show()
```

- 특정 칸만 불량률이 높으면 '두 조건이 동시에 만족될 때 불량' 이라는 상호작용 근거가 된다.

- qcut(..., duplicates='drop'): 같은 값이 많아 구간이 겹치면 자동으로 합친다 (그래서 4구간보다 적을 수 있음).

[수정 포인트]
- 구간 수는 qcut 의 4 를 3~5 로 조정. 불량이 적어 구간을 너무 잘게 나누면 칸마다 0/1건이 되어 의미가 없다.

[셀 30] 3-4. 판정 유형 집계 (TP/FN/FP/TN)

목적 개발셋 OOF 예측(학습에 안 쓰인 fold 에서 나온 예측)으로 각 행을 4가지 유형으로 분류한다.

```
oof["error"] = oof["error"].fillna("OK")
oof["type"] = np.select(
    [(oof[TARGET] == 1) & (oof.error == "OK"), oof.error == "FN", oof.error == "FP"],
    ["TP", "FN", "FP"], "TN")
display(pd.crosstab([oof["product"], oof["cavity"]], oof["type"]))
```

- TP 불량→불량(적중), FN 불량→양품(미탐지), FP 양품→불량(오경보), TN 양품→양품.
- OOF 를 쓰는 이유: 학습에 쓴 데이터로 오류를 분석하면 오류가 거의 안 보인다.

[출력 확인]
- (제품, 캐비티)별로 FN·FP 가 어디에 몰렸는지 본다.

[셀 31] 3-5. 유형별 공정변수 평균 히트맵

목적 TP/FN/FP/TN 그룹마다 공정변수 평균을 비교한다.

```
prof = oof.groupby("type")[use].mean().loc[["TP", "FN", "FP", "TN"]]
plt.figure(figsize=(13, 3.2))
sns.heatmap(prof, cmap="RdBu_r", center=0, annot=True, fmt=".1f", cbar=False)
plt.title("판정 유형별 공정변수 평균 (표준화 단위)"); plt.tight_layout(); plt.show()
```

- FN 이 TP 와 비슷하면 → 모델이 구분할 신호는 있는데 임계값이 높다. FN 이 TN 과 비슷하면 → 공정값상 양품과 구분 불가(라벨 충돌).
- FP 가 TP 와 비슷하면 → '불량처럼 보이는 양품' 조건. 보고서 3장 FP 분석의 핵심.

[셀 32] 3-6. 변수 구간별 FN·FP 비율

목적 중요 변수 5개 각각을 4구간으로 나눠 구간별 미탐지율·오경보율을 계산한다.

```
# 오류가 집중되는 공정조건: 변수 구간별 FN / FP 비율
def error_rate_by_bin(col, q=4):
    b = pd.qcut(oof[col], q, duplicates="drop")
    g = oof.groupby(b)
    return pd.DataFrame({
        "n": g.size(),
        "불량 수": g[TARGET].sum(),
        "FN 비율(불량 중)": g.apply(lambda d: (d.type == "FN").sum() / max(1, d[TARGET].sum())),
        "FP 비율(양품 중)": g.apply(lambda d: (d.type == "FP").sum() / max(1, (d[TARGET] == 0).sum())),
    }).round(3)

for c in imp.index.drop(["is_rg3", "cavity"], errors="ignore")[:5]:
    print(f"■ {c}")
    display(error_rate_by_bin(c))
```

- FN 비율(불량 중) = 해당 구간 불량 중 놓친 비율. FP 비율(양품 중) = 해당 구간 양품 중 잘못 경보한 비율.
- 특정 구간에 비율이 튀면 '이 조건에서 모델이 실패한다' 로 보고서에 적는다.

[출력 확인]
- 불량 수가 0~1 인 구간의 FN 비율은 의미가 약하니 '불량 수' 컬럼과 함께 본다.

[수정 포인트]
- 분석 변수 개수는 [:5], 구간 수는 error_rate_by_bin(c, q=4) 의 q 로 조정.

[셀 33] 3-7. 생산 순번별 OOF 불량확률

목적 시간 순서대로 예측확률을 찍고 TP/FN/FP/TN 을 색으로 구분한다. 점선 = 임계값.

```
# 생산 순번에 따른 예측확률 - 오류가 특정 시기에 몰리는지
fig, axes = plt.subplots(2, 1, figsize=(12, 5))
for ax, (p, d) in zip(axes, oof.groupby("product")):
    for t, col in [("TN", "#cccccc"), ("FP", "orange"), ("TP", "green"), ("FN", "red")]:
        s = d[d.type == t]
        ax.scatter(s[INDEX_COL], s["oof_prob"], s=10 if t == "TN" else 30, c=col, label=t)
    ax.axhline(THR, ls="--", c="k", lw=.8)
    ax.set_title(f"{p.upper()} - 생산 순번별 OOF 불량확률"); ax.legend(ncol=4, fontsize=8)
plt.tight_layout(); plt.show()
```

- CN7 에서 FN 이 특정 시기에 몰리면 '시기성 사건(설비 이상 등)을 공정변수가 다 설명하지 못함' 으로 해석.
- FP 가 불량 구간 주변에 몰리면 '공정 이상 구간은 잡지만 양품까지 함께 경보' 로 해석 → 현장에서는 구간 단위 점검으로 연결 가능.

[셀 34] 3-8. 운전구간 군집 — 라벨 없이 찾은 불량 다발 구간

목적 공정값만으로 K-평균 군집을 만들고 군집별 불량률과 생산순번 구간을 비교한다.

```
rc = pd.read_csv(OUT_DIR / "regime_clusters.csv")
rc["fail_rate(%)"] = (rc["fail_rate"] * 100).round(2)
display(rc[["product", "cluster", "rows", "fails", "fail_rate(%)", "idx_start", "idx_end", "k", "silhouette"]].r
    ound(3))

asg = pd.read_csv(OUT_DIR / "regime_cluster_assign.csv")
fig, axes = plt.subplots(2, 1, figsize=(12, 4))
for ax, (p, d) in zip(axes, asg.groupby("product")):
    ax.scatter(d[INDEX_COL], d["cluster"], c=d["cluster"], cmap="tab10", s=4)
    f = d[d[TARGET] == 1]
    ax.scatter(f[INDEX_COL], f["cluster"], marker="x", c="red", s=40, label="불량")
    ax.set_title(f"{p.upper()} - 생산 순번별 운전구간 군집 (x = 불량)"); ax.set_ylabel("군집"); ax.legend()
plt.tight_layout(); plt.show()
```

- **K-평균**: 비슷한 공정값끼리 k 개 묶음으로 나누는 비지도 방법. 라벨을 전혀 쓰지 않는다.
- **k 선택**: 2~6 중 실루엣 점수가 가장 높은 k 를 고른다. 실루엣 점수는 같은 군집끼리 얼마나 가깝고 다른 군집과 얼마나 먼지를 -1~1 로 나타낸 값(0.3 안팎이면 '경향은 있다' 수준).
- 그래프: x = 생산 순번, y = 군집 번호, 빨간 x = 불량. 군집이 순번 구간별로 나뉘면 '운전 상태(구간)' 를 찾은 것이다.
- **보고서 메시지**: 라벨 없이 만든 군집에서도 CN7 초반 구간의 불량률이 약 4배 → '가동 초기 강화검사' 규칙의 독립 근거.

[출력 확인]
- CN7(k=3): 순번 0~374 군집 불량률 2.98%, 이후 구간 0.72%, 마지막 4행은 별도 군집(불량 0).
- RG3(k=2): 전반부 2.56%, 후반부 1.73% — 차이가 작고 구간이 겹침.

[수정 포인트]
- 군집 수 후보는 src/experiments.py 의 regime_clusters(k_range=range(2, 7)) 에서 조정.

4장. 현장 활용방안 (보고서 4장 · 10점)

[셀 35] 4-1. 검사 우선순위 이득곡선

목적 불량확률이 높은 순으로 검사할 때, 검사 비율(x) 대비 누적 불량 포착률(y)을 그린다.

```
def gain_curve(y, p):
    order = np.argsort(-p)
    return np.arange(1, len(y) + 1) / len(y), np.cumsum(y[order]) / y.sum()

plt.figure(figsize=(6, 4.5))
for p, d in oof.groupby("product"):
    x, r = gain_curve(d[TARGET].to_numpy(), d["oof_prob"].to_numpy())
    plt.plot(x * 100, r * 100, label=p.upper())
plt.plot([0, 100], [0, 100], "--", c="gray", label="무작위 검사")
for k in (5, 10, 20):
    plt.axvline(k, c="lightgray", lw=.8)
plt.xlabel("검사 비율 (%)"); plt.ylabel("불량 포착률 (%)"); plt.title("검사 우선순위 이득곡선 (dev OOF)")
plt.legend(); plt.tight_layout(); plt.show()
```

- 점선(대각선) = 무작위로 검사할 때. 곡선이 왼쪽 위로 치솟을수록 적게 검사하고 많이 잡는다.
- 세로선 5/10/20% 는 현장 검사 여력 시나리오.

[출력 확인]
- CN7 곡선이 RG3 보다 훨씬 위에 있어야 정상.

[셀 36] 4-2. 검사 비율별 포착률 표

목적 검사 비율 5/10/20/30% 에서 몇 개를 검사해 불량 몇 개를 잡는지 표로 정리한다.

```
rows = []
for p, d in oof.groupby("product"):
    y, s = d[TARGET].to_numpy(), d["oof_prob"].to_numpy()
    for k in (0.05, 0.10, 0.20, 0.30):
        n = int(np.ceil(len(y) * k)); top = np.argsort(-s)[:n]
        rows.append({"product": p, "검사비율": f"{int(k*100)}%", "검사 수": n,
                     "포착 불량": int(y[top].sum()), "전체 불량": int(y.sum()),
                     "포착률": round(y[top].sum() / y.sum(), 3),
                     "무작위 대비 배수": round((y[top].sum() / y.sum()) / k, 2)})
pd.DataFrame(rows)
```

- '무작위 대비 배수' = 포착률 ÷ 검사비율. 예: 20% 검사로 80% 포착 → 4배 효율.
- 보고서 4장 '현장 활용'의 핵심 표. '전수검사 대신 상위 20% 만 검사해도 불량의 X% 확보' 식으로 쓴다.

[수정 포인트]
- 검사 시나리오는 (0.05, 0.10, 0.20, 0.30) 튜플을 바꾼다.

[셀 37] 4-3. 위험등급 3단계

목적 예측확률 분위수로 정상/주의/위험 등급을 나누고 등급별 실제 불량률을 확인한다.

```
# 위험등급 제안: 확률 분위 기반 3단계
q_hi, q_mid = np.quantile(oof["oof_prob"], [0.90, 0.70])
oof["risk"] = pd.cut(oof["oof_prob"], [-np.inf, q_mid, q_hi, np.inf], labels=["정상", "주의", "위험"])
display(oof.groupby("risk", observed=True).agg(n=(TARGET, "size"), 불량=(TARGET, "sum"),
                                               불량률=(TARGET, "mean")).round(3))
print(f"경계값 – 주의 ≥ {q_mid:.3f}, 위험 ≥ {q_hi:.3f}")
```

- 현재 기준: 상위 10% = 위험, 상위 10~30% = 주의, 나머지 = 정상.
- 등급이 올라갈수록 실제 불량률이 뚜렷하게 커져야 등급 체계가 의미 있다.

[출력 확인]
- 출력된 경계값(확률)을 현장 규칙으로 제시: '확률 ≥ x 이면 즉시 검사'.

[수정 포인트]
- 등급 기준은 np.quantile(..., [0.90, 0.70]) 의 두 숫자로 조정.

5장. 제출용 예측결과와 원인 인자

[셀 38] 5-1. 테스트 예측결과와 제품별 원인 인자

목적 run_all.py 가 저장한 제출물 '테스트데이터 예측결과 파일'(outputs/test_predictions.csv)을 불러와 확인한다.

```
pred = pd.read_csv(OUT_DIR / "test_predictions.csv")
assert np.allclose(pred.set_index(INDEX_COL).loc[test[INDEX_COL], "fail_prob"].values, p_test)  # 노트북 모델과 동일 확인
pred.head(15)
```

- 컬럼: 원본 순번, 제품, 사출번호, 캐비티, 실제 정답, fail_prob(불량확률), pred(판정 0/1), inspect_rank(검사 우선순위 1=가장 먼저), **factor_1~3**(원인 인자).
- **원인 인자**: 그 제품의 SHAP 값 중 양(+)의 기여가 가장 큰 변수 3개와 기여도. 예: cavity(+0.049) = 캐비티 0 이라서 불량확률이 0.049 올라갔다.
- assert 줄: 노트북에서 다시 학습한 최종모델의 확률이 파일과 똑같은지 확인한다. 다르면 에러가 나므로 노트북과 run_all.py 의 결과 일치를 보장한다.
- 예측 파일은 이제 노트북이 아니라 run_all.py 만 저장한다 (원인 인자 계산이 그쪽에 있으므로).

[출력 확인]
- 검사 순위 1·2위(CN7 같은 사출의 두 제품, 둘 다 불량)는 Max_Injection_Speed 가 원인 1순위 → 캐비티와 무관한 공정 이상.
- 그 외 상위 제품은 대부분 cavity 가 1순위.

[수정 포인트]
- 원인 인자 개수는 src/experiments.py 의 top_factors(model, X, k=3) 의 k 로 조정.

[셀 39] 5-2. 상위 20% 제품의 원인 인자 빈도

목적 검사 우선순위 상위 20% 제품에서 factor_1~3 에 가장 자주 등장한 변수를 센다.

```
# 검사 우선순위 상위 제품에서 가장 자주 등장한 원인 인자
top = pred[pred["inspect_rank"] <= int(len(pred) * 0.2)]
fac = (pd.concat([top[c] for c in ["factor_1", "factor_2", "factor_3"]])
         .replace("", np.nan).dropna().str.replace(r"₩(.*₩)", "", regex=True))
fac.value_counts().head(8).rename("상위 20% 제품의 원인 인자 등장 횟수")
```

- str.replace(r'₩(.*₩)', '') 로 '(+0.049)' 같은 기여도 표시를 지우고 변수명만 남긴 뒤 value_counts() 로 센다.
- **현장 해석**: cavity 가 많으면 금형 캐비티 점검, 공정변수가 많으면 해당 공정조건 점검을 우선한다. 보고서 4장 '제품별 원인 인자 제시' 의 근거.

[출력 확인]
- cavity 85, is_rg3 74, Clamp_Close_Time 46, Plasticizing_Position 16, Cycle_Time 9 순.

[수정 포인트]
- is_rg3 는 제품 구분 변수라 조치 대상이 아니므로, 보고서에는 공정변수와 cavity 위주로 인용한다.

부록 1. 용어 정리

용어	뜻	이 프로젝트에서
사출(shot) / 캐비티	금형에 수지를 한 번 주입하는 것 / 금형 안의 제품 자리	1회 사출에 제품 2개, 불량 86% 가 캐비티 0
그룹 CV	묶음(그룹)이 train/valid 에 쪼개지지 않게 나누는 교차검증	그룹 = 제품_사출번호
OOF 예측	해당 행이 학습에 안 쓰인 fold 에서 나온 예측	오류분석(3장)의 입력
PR-AUC	정밀도-재현율 곡선 아래 면적. 무작위 = 불량률	주지표. 무작위 약 0.018
F1	정밀도와 재현율의 조화평균 (임계값 필요)	학습 fold 내부에서 임계값 선정
Recall@k%	확률 상위 k% 만 검사할 때 잡는 불량 비율	검사 우선순위 지표
FN / FP	미탐지(불량→양품) / 오경보(양품→불량)	3장 오류분석 대상
SHAP	각 변수가 예측을 얼마나 올리고 내렸는지 분해	영향변수·상호작용 근거
확률 보정	예측확률이 실제 비율과 맞도록 조정	창의성 항목 재료
데이터 누수	평가 데이터의 정보가 학습에 새어 들어가 성능이 부풀려지는 것	행 단위 CV 시 최대 +43%
시간 누수 / 시간 구간 검증	같은 시기의 데이터가 학습·평가에 섞여 그 시기를 외우는 것 / 연속 구간을 통째로 빼서 검증	처음 보는 구간 PR-AUC 0.05~0.06 → 재학습 필수
배깅 / 부스팅	나무를 독립적으로 여러 개 만들어 평균 / 앞 나무의 오류를 다음 나무가 순차 보완	소수 불량 데이터에서 배깅 우세
유의성 검정	두 모델 차이가 우연인지 판단. Wilcoxon(짝 비교), 보정 t(반복 CV 겹침 보정)	모델 차이는 비유의, 캐비티 효과는 유의
SMOTE / ADASYN	소수 클래스 사이를 보간해 합성 표본을 만드는 오버샘플링	모두 PR-AUC 하락 → 미채택
도메인 판별	두 데이터를 구분하는 분류기의 AUC 로 분포 차이를 재는 방법	라벨/비라벨 AUC 1.00
자기학습	모델 예측을 가짜 라벨로 붙여 비라벨 데이터를 학습에 추가하는 준지도 방법	효과 불안정 → 미채택
오토인코더	입력을 압축·복원하도록 학습한 신경망. 복원 오차 = 이상 점수	양품 학습, 보조 경보용
K-평균 / 실루엣	비슷한 것끼리 k 개로 묶는 군집 / 묶음이 얼마나 뚜렷한지 점수	CN7 초반 운전구간 분리
원인 인자	제품별 SHAP 양의 기여 상위 변수	test_predictions 의 factor_1~3

부록 2. 자주 나는 오류

증상	원인 · 해결
ModuleNotFoundError: No module named 'src'	노트북을 injection_molding 폴더가 아닌 곳에서 열었다. 폴더에서 jupyter 실행.
numpy / pandas DLL load failed	miniconda 기본 환경 문제. anaconda3 커널로 바꾼다.
그래프 한글이 □ 로 깨짐	font.family 가 설치되지 않은 글꼴. Windows = 'Malgun Gothic'.
SHAP 셀이 너무 오래 걸림	X_dev.sample(500, random_state=0) 로 표본만 설명.
설정을 바꿨는데 2장 결과가 그대로	outputs/cv_folds.csv 를 지우고 셀 0-2 재실행 (또는 python run_all.py).
qcut 에서 에러 / 구간이 적게 나옴	같은 값이 많은 변수. duplicates='drop' 유지, 구간 수 q 를 줄인다.

증상	원인 · 해결
KAMP-NOTE: Permission denied 'METADATA'	시스템 패키지(/opt/conda) 교체 권한이 없다. pip install --user -r requirements.txt 로 설치 (requirements 는 버전 미고정).
ModuleNotFoundError: No module named 'shap'	pip install --user shap 후 커널 재시작. run_all.py 도 원인 인자 계산에 shap 을 쓴다.
ModuleNotFoundError: xgboost / catboost / imblearn	pip install --user -r requirements.txt (xgboost, catboost, imbalanced-learn 포함) 후 커널 재시작.
FileNotFoundError: leakage_cv.csv 등	RUN_EXTRA = False 로 실행해 부가 실험 파일이 없다. True 로 두고 python run_all.py 재실행.
AssertionError (셀 5-1)	outputs/test_predictions.csv 가 현재 코드와 다른 실행 결과. python run_all.py 로 다시 만든다.

부록 3. 제출 전 체크리스트

- 새 가상환경(또는 KAMP-NOTE)에서 pip install (--user) -r requirements.txt → python run_all.py 가 에러 없이 끝나는가
- outputs/ 에 significance.csv, imbalance_cv.csv 등 부가 실험 결과까지 모두 생성되었는가
- 노트북을 '전체 재시작 후 모두 실행' 해서 출력이 저장된 상태인가
- 출력·README·파일명에 학교명/소속/절대경로(C:₩Users₩...)가 없는가 (블라인드 심사)
- outputs/test_predictions.csv 가 zip 에 포함되었는가
- 보고서의 수치가 cv_summary.csv / test_metrics.json 과 일치하는가
- 이 해설지 PDF 는 제출 zip 에서 제외했는가

부록 4. 기초 개념 정리

해설지를 읽다가 막히는 개념을 문제 → 데이터 구조 → 모델 → 검증 → 지표 → 해석 → 부가 실험 → 현장 활용 → 코드 구조 순서로 정리했다. 이 순서는 노트북 흐름과 같으므로, 처음 읽는 사람은 이 부록을 먼저 훑어본 뒤 0장부터 보면 수월하다. 각 절 제목 옆 괄호는 해설지에서 해당 개념이 주로 나오는 곳이다.

1. 이 프로젝트가 풀려는 문제 (0장·1장)

사출기가 한 번 돌 때마다 기록되는 공정변수 24개(온도, 압력, 시간, 위치 등)를 보고, 그 제품이 양품(0)인지 불량(1)인지 미리 맞히는 것이 목표다. 정답이 둘 중 하나이므로 이진 분류 문제라고 한다. 정답 컬럼(TARGET)을 라벨이라 부르고, 라벨이 있는 데이터(약 2,400행)는 학습에 쓰며, 라벨이 없는 데이터(약 7만 행)는 비라벨 데이터라 부른다.

제품은 CN7과 RG3 두 종류이고, 모델에는 is_rg3(RG3면 1, CN7이면 0) 변수를 넣어 어떤 제품인지 알려준다. 단순히 '불량이다/아니다'만 맞히는 것이 아니라, 불량확률을 계산해 위험한 것부터 검사하자(검사 우선순위)는 것이 과제의 핵심 방향이다.

2. 사출(shot)과 캐비티(cavity) 복원 (A장 data.py · 1-3)

사출은 녹인 플라스틱을 금형에 쏘아 넣어 굳히는 공정이고, 캐비티는 금형 안에서 제품 모양이 만들어지는 빈 공간이다. 이 금형은 2캐비티라서 한 번 사출하면 제품이 2개 나온다. 그런데 데이터에는 몇 번째 사출인지, 어느 캐비티인지가 적혀 있지 않아 add_shot_structure 함수가 이를 추정해 만든다.

원리: 같은 사출의 두 제품은 공정값이 완전히 똑같이 기록된다. 그래서 바로 윗줄과 공정값이 하나라도 다르면 '새 사출 시작'으로 보고 shot_id 를 1 올리고, 사출 안의 첫 행은 cavity=0, 두 번째 행은 cavity=1 을 붙인다. 이때 '행 순서 = 캐비티 순서'라는 가정이 들어가므로 보고서에 반드시 가정으로 밝힌다.

이것이 중요한 이유: 전체 불량 42건 중 36건(약 86%)이 캐비티 0에서 나왔다. 금형의 특정 자리 자체가 불량과 관련 있다는 뜻이다. 이 정보를 모델에 넣은 특성셋이 process+cavity, 넣지 않은 것이 process_only 다.

주의: 일반적인 전처리처럼 '완전히 똑같은 행'을 중복으로 지우면 2캐비티 짝 중 한 행이 사라지면서 불량 라벨까지 함께 지워진다. 이 데이터에서는 중복을 지우지 않고 사출 단위로 묶는다.

3. 데이터 진단 용어 (1-1 ~ 1-10)

용어	뜻	이 프로젝트에서
불량률	전체 중 불량의 비율	CN7 1.40%, RG3 2.12%
결측치	비어 있는 값	0개
상수 컬럼	모든 행의 값이 같아 정보가 없는 변수	Clamp_Open_Position → 제거
표준화	변수마다 단위가 달라 평균 0, 표준편차 1로 맞춘 것. '표준화 단위 +1' = 표준편차 1개만큼 큼	라벨/비라벨 데이터가 각자 따로 표준화돼 있어 같은 숫자가 같은 실제 값이 아닐 수 있음 → 1-6-2 에서 결합 가능성을 실험으로 확인
라벨 충돌	공정값 24개가 완전히 같은데 양품·불량이 섞인 경우	RG3 불량 25건 전부 해당 → 공정값만으로는 원리적으로 구분 불가, 캐비티 정보 필요
상관	두 변수가 함께 움직이는 정도	배럴 온도 1~6처럼 상관 높은 변수는 사실상 같은 정보 → 해석 시 묶음으로 봄

4. 클래스 불균형과 정확도의 함정 (1-2)

불량이 1~2%밖에 없는 상태를 클래스 불균형이라 한다. 이때 모델이 무조건 '전부 양품'이라고 찍어도 정확도(Accuracy)는 98%가 나온다. 불량은 하나도 못 잡았는데 점수는 높은 것이다. 그래서 이 프로젝트는 정확도를 쓰지 않고 PR-AUC, F1, Recall@k% 를 쓴다. 또 모델이 소수인 불량을 무시하지 않도록 불량 클래스에 더 큰 비중을 주는데, 이를 클래스 가중치(불균형 가중)라 한다.

5. 비교한 모델 (A장 models.py · 2-1 · 2-8)

모델	작동 방식	역할
LR_baseline (로지스틱 회귀)	변수들의 가중합으로 확률을 내는 가장 단순한 모델	베이스라인(비교 기준점). 복잡한 모델이 얼마나 나은지 보여주는 잣대
RandomForest	'이 변수가 이 값보다 크면 왼쪽' 식의 의사결정나무를 수백 그루 만들어 투표 (배깅)	비교 모델 (PR-AUC 0.114)
ExtraTrees	RandomForest 와 같지만 분할 기준값까지 무작위로 정해 나무들이 더 다양함	최종모델 (+캐비티, PR-AUC 0.139)
LightGBM · XGBoost · CatBoost	나무를 하나씩 추가하며 앞 나무가 틀린 부분을 다음 나무가 보완 (부스팅)	비교 모델 (0.098~0.110)
RF+ET 앙상블	두 모델의 확률을 평균 (소프트 보팅)	비교 모델 (0.130)
스태킹 앙상블	여러 모델의 예측을 메타 모델(로지스틱 회귀)이 다시 조합	검토 후 미채택 (0.127)
MLP (딥러닝)	은닉층이 있는 신경망. 수많은 가중치를 학습	검토 후 미채택 (0.091~0.099, 베이스라인보다 낮음)
AutoEncoder (비지도)	양품만 보고 압축·복원을 학습한 신경망. 복원 오차가 크면 이상	비교·보조 경보 (라벨 미사용)

지도학습 모델 7개 × 특성셋 2개 = 14개 조합을 비교해, CV PR-AUC 가 가장 높은 조합을 최종모델로 고른다. 오토인코더는 라벨을 쓰지 않는 다른 방식의 대안으로 같은 fold 에서 따로 비교한다. 불량이 34건뿐이라 오분류 사례에 계속 집중하는 부스팅은 과적합하기 쉽고, 분할을 무작위로 정하는 ExtraTrees 가 가장 안정적이었다. 불량이 1만 건인 선행연구에서는 부스팅이 우세했으므로, 적합한 모델 계열은 표본 규모에 따라 다르다.

6. 검증 방식 — 이 프로젝트의 핵심 (A장 evaluate.py · 2-4 · 2-7)

학습 / 검증 / 테스트 — 모델은 이미 본 데이터는 당연히 잘 맞히므로, 학습에 쓰지 않은 데이터로 채점해야 진짜 실력을 알 수 있다. 그래서 먼저 개발셋(dev) 80%와 테스트셋(test) 20%로 나눈다(holdout_split). 테스트셋은 마지막에 한 번만 꺼내 보는 '최종 시험지'다. 테스트 불량이 8건뿐이라 수치가 크게 흔들리므로 보고서의 주 근거는 CV 결과다.

교차검증(CV)과 fold — 개발셋 안에서 모델을 비교할 때 한 번만 나눠 채점하면 운에 따라 결과가 흔들린다. 그래서 개발셋을 5조각(fold)으로 나눠 4조각으로 학습하고 1조각으로 채점하는 것을 5번 돌린다(5-fold CV). 나누는 방식을 바꿔 이를 5번 반복하므로 모델 하나당 25번 채점한다. 결과를 '평균 ± 표준편차'로 보여주는 이유다.

StratifiedGroupKFold = 층화 + 그룹 — Stratified(층화): 각 fold 에 불량 비율이 비슷하게 들어가도록 나눈다. 불량이 적어 어떤 fold 에 불량이 0개가 되는 것을 막는다. Group(그룹): 같은 사출의 두 제품은 공정값이 똑같다. 하나는 학습에, 하나는 채점에 들어가면 모델은 이미 본 문제를 다시 푸는 셈이 된다. 이처럼 채점 데이터의 정보가 학습에 새어 들어가는 것을 데이터 누수(leakage)라 하며 성능이 실제보다 부풀려진다. 그래서 같은 사출(group = 제품_사출번호)은 반드시 한쪽으로만 가도록 묶어서 나눈다.

누수는 실제로 얼마나 부풀리나 — 2-7 셀에서 같은 모델을 행 단위로 섞어 검증하면 PR-AUC 가 LR 은 32%, RandomForest 는 36%, 최종모델 ExtraTrees 는 43% 높게 나왔다. 그룹 CV 가 '깐깐한' 대신 현장 성능에 가까운 숫자다.

OOF(Out-Of-Fold) 예측 — CV 를 돌리면 개발셋의 모든 행이 한 번씩은 채점용으로 쓰인다. 이때 나온 예측, 즉 그 행을 학습에 쓰지 않은 모델이 낸 예측을 모은 것이 OOF 예측(oof_dev.csv)이다. 3장 오류분석과 4장 현장 활용은 모두 이것을 쓴다. 학습에 쓴 데이터로 분석하면 오류가 거의 보이지 않기 때문이다.

7. 임계값(threshold) (A장 best_threshold · 2-4)

모델은 '불량일 확률 0.23'처럼 확률을 낸다. 이를 0/1 판정으로 바꾸려면 '몇 이상이면 불량으로 볼까'라는 기준선이 필요한데 이것이 임계값이다. 불량이 드물어서 보통 0.5보다 훨씬 낮은 값이 최적이다.

이 프로젝트는 F1 이 가장 높은 임계값을 고르는데, 채점용 데이터를 보고 고르는 것도 누수다(시험지를 보고 커트라인을 정하는 것과 같다). 그래서 학습 데이터 안에서 다시 작게 CV 를 돌려(inner_oof) 그 결과로만 임계값을

정한다.

8. 평가 지표 (2-1 ~ 2-8 · 4-1 ~ 4-2)

혼동행렬

	모델: 양품	모델: 불량
실제 양품	TN 정상 통과	FP 오경보 (괜히 검사하는 비용)
실제 불량	FN 미탐지 (불량 유출, 가장 치명적)	TP 적중

지표 한눈에 보기

지표	뜻	읽는 법
Precision (정밀도)	불량이라고 한 것 중 진짜 불량 비율 = TP / (TP+FP)	경보가 얼마나 믿을 만한가
Recall (재현율)	진짜 불량 중 잡아낸 비율 = TP / (TP+FN)	얼마나 놓치지 않는가
F1	Precision 과 Recall 의 조화평균	한쪽만 좋으면 낮게 나옴. 임계값에 따라 바뀜
ROC-AUC	임계값과 무관하게 불량을 위로 줄세우는 능력	불균형 데이터에서는 양품이 많아 점수가 후하게 나오는 경향
PR-AUC (주지표)	Precision-Recall 곡선 아래 면적. 불량 쪽 성능에 집중	무작위 = 불량률(약 0.018). 최종모델 0.139 는 무작위의 약 7.7배
Recall@k%	확률 상위 k% 만 검사했을 때 전체 불량 중 잡는 비율	Recall@20% = 0.8 이면 20% 검사로 불량 80% 확보 → 무작위의 4배

PR-AUC 는 절댓값보다 '무작위 대비 몇 배인가'로 읽는다. 0.139 가 낮아 보여도 불량률 1.8% 인 데이터에서는 아무렇게나 찍을 때보다 약 7.7배 불량을 잘 골라낸다는 뜻이다. 단, 한 지표만 보면 안 된다. 오토인코더는 통합 PR-AUC 가 0.123 으로 더 높지만 Recall@20% 는 0.281 로 최종모델(0.707)의 40% 수준이라, 실제 검사 운영에는 최종모델이 낫다.

이득곡선(gain curve) — 검사 비율을 0~100% 로 늘려가며 Recall@k% 를 이은 곡선이다. 대각선(무작위 검사)에서 왼쪽 위로 멀수록 적게 검사하고 많이 잡는다는 뜻이다.

확률 보정(calibration) — 모델이 '30%'라고 한 제품들 중 실제로 30% 가 불량이면 잘 보정된 것이다. 랜덤포레스트 같은 모델은 확률이 한쪽으로 쏠리는 경우가 많아, 확률 값 자체를 현장 규칙에 쓰려면 보정 곡선으로 확인하고 필요하면 Platt(sigmoid) / Isotonic 방법으로 조정한다.

9. 해석: SHAP, 원인 인자, 오류분석 (3장 · 5-1)

SHAP 은 '이 제품의 불량확률이 평균보다 높게 나온 이유'를 변수별로 쪼개 보여준다. 예: 'cavity=0 이라서 +0.05, 사출압력이 높아서 +0.02'. 모든 제품의 SHAP 크기를 평균하면 변수 중요도가 되고, 변수 값에 따라 SHAP 이 어떻게 변하는지 그린 것이 의존도 그래프(dependence plot)다.

원인 인자는 SHAP 을 제품 하나 단위로 쓴 것이다. 그 제품에서 양(+)의 기여가 가장 큰 변수 3개를 factor_1~3 으로 저장한다. cavity 가 1순위면 금형 점검, 사출속도 같은 공정변수가 1순위면 공정조건 점검처럼, 예측 결과를 구체적인 조치로 연결하는 역할을 한다.

상호작용은 'A 만 높을 땐 괜찮은데 A 와 B 가 동시에 높으면 불량'처럼 두 조건이 겹칠 때 생기는 효과다. 3-3 의 구간별 불량률 히트맵이 이를 확인하는 용도다. 여기서 사분위(qcut 4구간)는 값을 크기순으로 줄세워 같은 개수씩 4묶음으로 나누는 것이다.

오류분석은 OOF 예측을 TP/FN/FP/TN 으로 나눠 '어떤 조건에서 모델이 틀리는가'를 보는 것이다. FN(놓친 불량)의 공정값이 TN(정상 양품)과 거의 같다면 모델 탓이 아니라 공정 데이터에 구분할 정보가 없다는 뜻이고, 1장의 라벨 충돌과 연결된다.

10. 부가 실험: 비지도 · 준지도 · 군집 (1-6-2 · 2-7 · 2-8 · 3-8)

지도 / 비지도 / 준지도 — 지도학습은 정답(라벨)을 보고 배우는 방식(LR, RF, LightGBM), 비지도학습은 정답 없이 데이터 모양만 보고 배우는 방식(오토인코더, K-평균), 준지도학습은 적은 라벨과 많은 비라벨을 함께 쓰는

방식(자기학습)이다. 제조AI 기술 동향에서도 세 가지가 모두 주요 키워드로 나온다.

도메인 판별 — 두 데이터를 섞어 놓고 '이 행은 어느 데이터에서 왔나'를 맞히는 분류기를 만든다. 맞히기 어려우면(AUC 0.5) 두 데이터는 비슷하고, 쉽게 맞히면(AUC 1.0) 완전히 다르다. 라벨/비라벨이 AUC 1.00 이었으므로 비라벨을 학습에 섞으면 다른 분포가 섞여 들어간다.

자기학습(가짜 라벨) — 라벨로 학습한 모델이 비라벨 데이터를 예측하고, 확신이 큰 것(확률 아주 높음/낮음)에 가짜 정답을 붙여 학습 데이터에 추가한다. 분포가 같으면 데이터가 늘어난 효과가 나지만, 분포가 다르면 틀린 가짜 라벨이 섞여 오히려 나빠질 수 있다. 실험 결과 효과가 제품·지표마다 엇갈려 채택하지 않았다.

K-평균 군집과 실루엣 점수 — K-평균은 비슷한 행끼리 k 개 묶음으로 나눈다. 실루엣 점수는 묶음이 얼마나 뚜렷한지(-1~1, 높을수록 뚜렷)를 나타내며 k 를 고르는 데 쓴다. 라벨 없이 만든 묶음이 CN7 초반 구간을 따로 분리하고 그 구간 불량률이 약 4배였으므로, '가동 초기에는 공정이 불안정하다'는 해석에 독립적인 근거가 생긴다.

11. 현장 활용 (4장 · 5장)

모델을 그대로 판정기로 쓰기보다 검사 우선순위 도구로 제안한다. 불량확률 순으로 줄세워 검사 여력(5%, 10%, 20%...)만큼 위에서부터 검사하는 방식이며, 제출 파일의 inspect_rank 가 이 순위다(1 = 가장 먼저). 각 제품의 factor_1~3 은 검사자가 무엇을 확인해야 하는지 알려준다.

위험등급은 확률 분위수로 나눈다. 상위 10% = 위험, 그다음 20% = 주의, 나머지 = 정상. 등급이 올라갈수록 실제 불량률이 뚜렷이 커져야 등급 체계가 의미 있다. 분위수란 전체를 줄세웠을 때 특정 비율 지점의 값이다(예: 90% 분위수 = 하위 90% 와 상위 10% 의 경계).

12. 코드 구조와 재현성 (0장 · 부록 3)

계산 로직은 src/ 폴더 5개 파일에 모아 두고, run_all.py(전부 실행해 결과 파일 저장)와 analysis.ipynb(결과를 그림·표로 보여주기)가 그 함수를 같이 불러 쓴다. 로직을 한 곳만 고치면 양쪽에 반영되는 구조다.

재현성은 다른 사람이 같은 코드를 돌려도 같은 결과가 나오는 것이다. 데이터를 무작위로 나누거나 나무를 만들 때 쓰는 난수의 출발점을 시드(SEED)로 고정해 두는 이유이며, 심사 항목(10점)에도 들어가 있다. requirements.txt 는 KAMP-NOTE 처럼 미리 설치된 패키지를 바꿀 권한이 없는 환경에서도 설치되도록 버전을 고정하지 않았고, 실제로 동작을 확인한 버전은 README 의 '검증 환경' 표에 적어 두었다.

> **한 줄 요약** — 해설지가 반복해서 강조하는 세 가지(사출 단위 그룹 CV, 학습 fold 안에서만 임계값 선정, PR-AUC + Recall@k%)는 모두 '성능을 부풀리지 않고, 현장에서 쓸 수 있는 방식으로 평가하자'는 한 가지 원칙에서 나온다. 부가 실험들은 그 원칙대로 다른 방법(비지도·준지도)도 시험해 본 뒤 데이터에 맞는 것만 채택했다는 근거다.